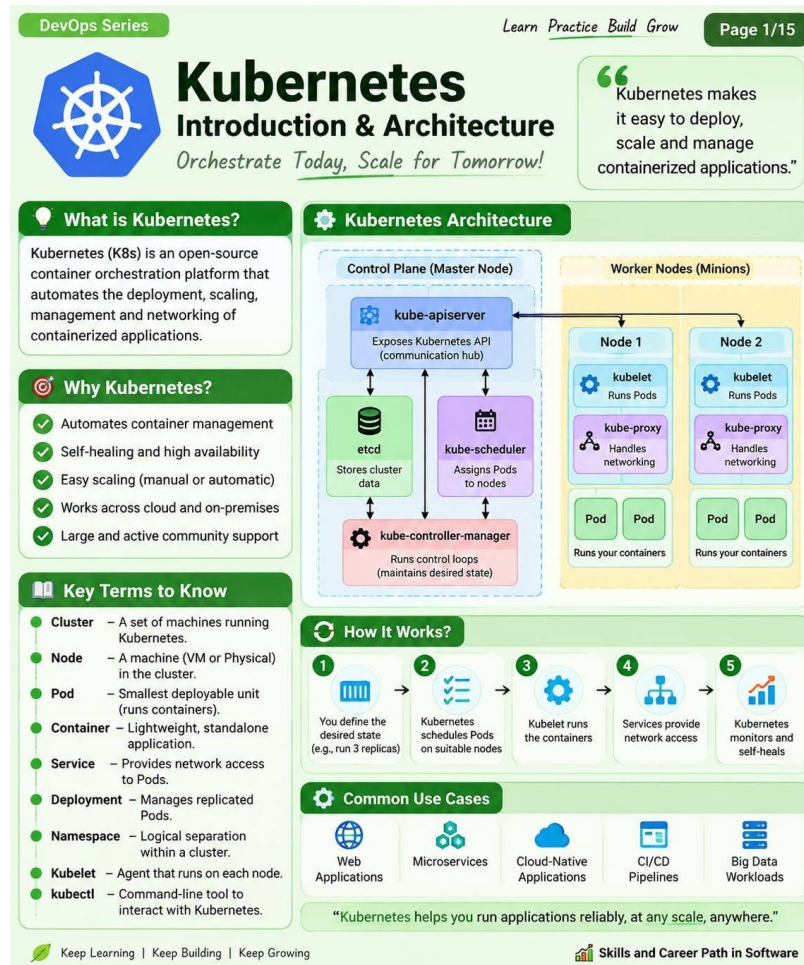


Kubernetes Reliability & Production Troubleshooting Handbook

Core concepts • failure detection • incident response • STAR playbooks



Purpose: A practical study and incident-response guide that connects Kubernetes fundamentals to production reliability. It combines the supplied visual references with deeper explanations, detection signals, remediation steps, and STAR-style incident narratives.

Prepared as a learning and SRE reference

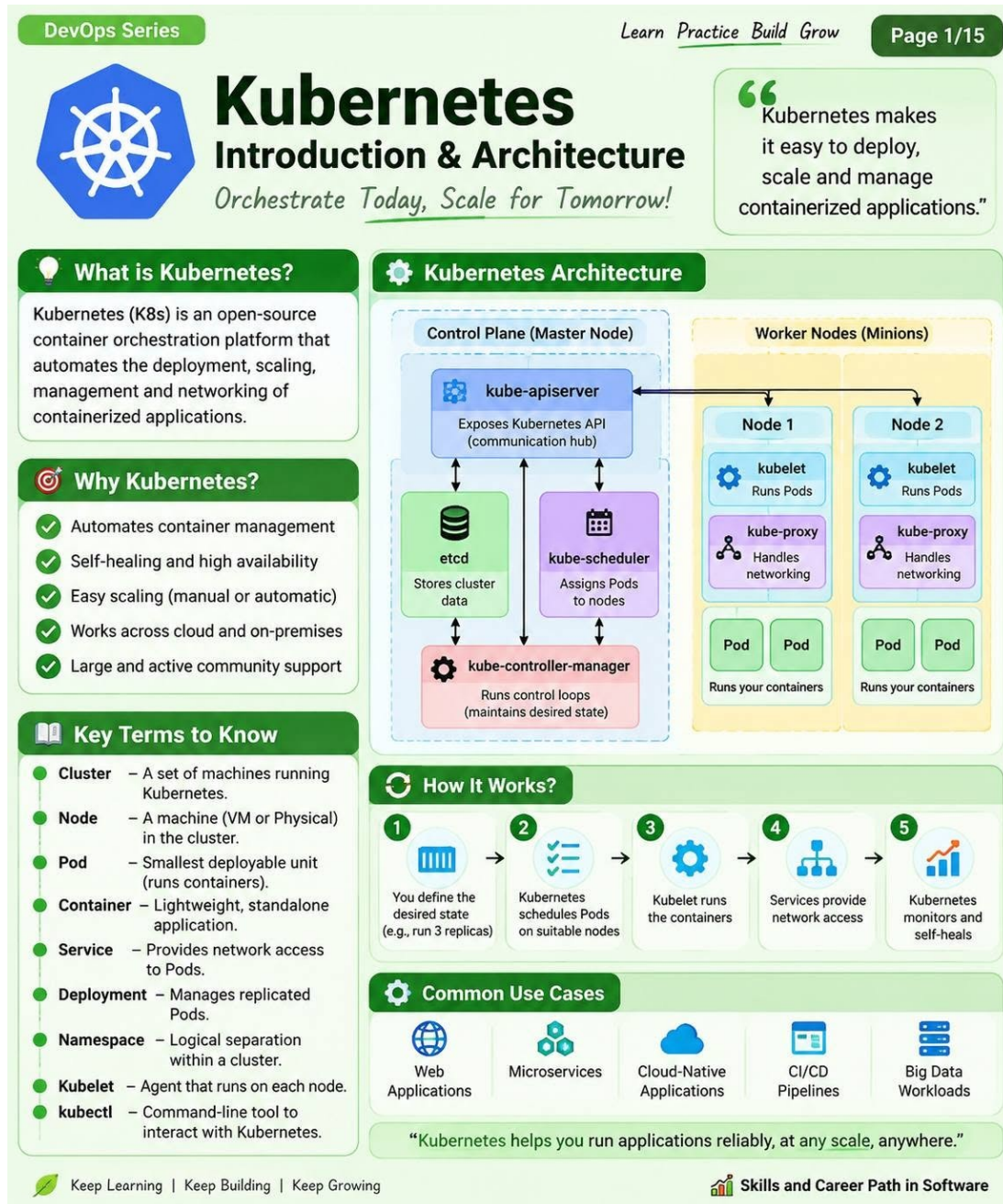
How to use this handbook

- **Learn the platform.** Sections 1-15 explain the building blocks shown in the supplied Kubernetes reference pages.
- **Think like an SRE.** Each concept is tied to production failure modes, observability, and reliability impact.
- **Troubleshoot systematically.** Start from symptoms, collect evidence, isolate the failing layer, mitigate, then verify.
- **Tell the story.** STAR examples use Scenario, Task, Action, Resolve so incident work can be explained clearly to managers or interviewers.

Reliability troubleshooting order

1. Symptom and blast radius: what is broken, for whom, and since when?
2. Events and object state: `kubectl get/describe`, rollout status, pod conditions.
3. Application evidence: current and previous container logs, probes, exit codes.
4. Service path: Service selectors, Endpoints/EndpointSlices, Ingress, DNS, NetworkPolicy.
5. Resource health: CPU, memory, storage, nodes, quotas, scheduling constraints.
6. Change correlation: deployment, config, secret, image, policy, or infrastructure change.
7. Mitigate first when impact is active; investigate root cause after service is stable.
8. Verify recovery and create prevention actions: alerting, tests, limits, runbooks, automation.

1. Introduction & Architecture



Kubernetes is a control system for desired state. You declare what should run; controllers continuously reconcile actual state toward that declaration.

Key production interpretation

Control plane: API server, etcd, scheduler, and controllers coordinate cluster state.

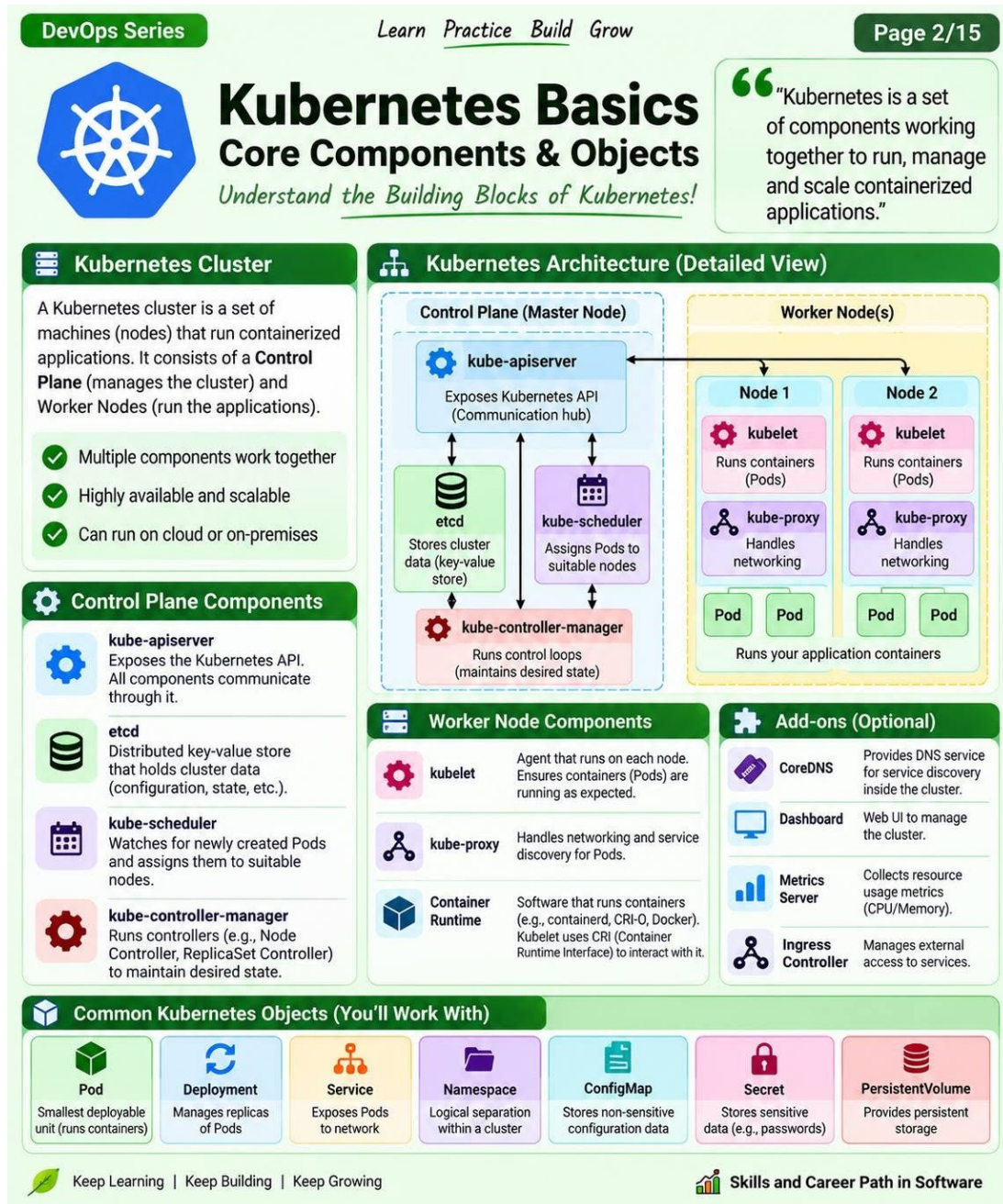
Worker nodes: kubelet runs pods; kube-proxy or CNI components implement service networking; the container runtime executes containers.

SRE angle: When something fails, ask whether the problem is control-plane state, scheduling, node/runtime health, application state, or network reachability.

Failure modes to recognize

- API server latency/unavailability prevents new changes and status updates.
- etcd health issues can impact the whole cluster.
- Scheduler problems leave Pods Pending.
- Kubelet/node failures surface as NodeNotReady, pod eviction, or stale status.

2. Core Components & Objects



Most production troubleshooting becomes easier when you know which component owns which responsibility.

Key production interpretation

API server: Front door for all Kubernetes API operations.

etcd: Source of truth for cluster configuration and state.

Scheduler: Chooses nodes for unscheduled pods.

Controller manager: Runs reconciliation loops for deployments, replicas, nodes, jobs, and more.

CoreDNS: Provides cluster DNS resolution.

Failure modes to recognize

- If kubectl itself cannot reach the cluster, start with API/auth/network rather than pods.
- If only one workload is broken, the control plane is less likely to be the root cause.
- If many namespaces fail at once, investigate shared cluster services first.

3. Pods

DevOps Series

Learn Practice Build Grow

Page 3/15



Kubernetes Basics

Pods

The Smallest Deployable Unit in Kubernetes!

“A Pod is the smallest unit in Kubernetes that can run one or more containers, sharing the same network and storage.”

What is a Pod?

A Pod is the smallest and simplest deployable unit in Kubernetes. It can run one or more containers that share the same:

- ✓ Network (same IP address)
- ✓ Storage (shared volumes)
- ✓ Lifecycle (created, started, stopped together)

Key Points

- ✓ Pods are ephemeral (temporary).
- ✓ Usually run one main container (1 container per Pod is a best practice).
- ✓ Multiple containers in a Pod share network and storage (sidecar pattern).
- ✓ Each Pod gets a unique IP address.
- ✓ Pods are scheduled on Worker Nodes by the Kubernetes scheduler.
- ✓ When a Pod is deleted, it is not recreated automatically (use Deployment for that).

Common Use Cases

- Running a single application container (e.g., Nginx, Python app)
- Sidecar containers (e.g., log collector, proxy)
- Init containers (run setup tasks before main container)
- Batch/Job processing

Pod Architecture

Pod (Single Node)

Shared Network Namespace (same IP address)

Container 1 (Main App) port: 80

Container 2 (Sidecar) Fluentd (Log Collector) port: 24224

Shared Volume (e.g., /var/log)

Runs on a Worker Node

Pod Lifecycle

Pending Pod is created but not yet scheduled.

Running Pod is running on a node.

Succeeded Pod completed successfully (e.g., for a Job).

Failed Pod terminated with an error.

Unknown Pod state cannot be determined.

Pod vs Container vs Deployment

	Container	A lightweight, standalone executable package.
	Pod	Smallest deployable unit. Can run one or more containers.
	Deployment	Manages and maintains multiple replicas of Pods.

Example: Simple Pod YAML

```
apiVersion: v1          # API version
kind: Pod               # Resource type
metadata:
  name: nginx-pod       # Pod name
  labels:
    app: nginx          # Label for identification
spec:
  containers:
    - name: nginx        # Container name
      image: nginx:latest # Docker image
      ports:
        - containerPort: 80 # Expose container port
```

Explanation

- Creates a Pod named nginx-pod
- Runs a single container with nginx:latest image
- Exposes port 80 inside the container
- You can add multiple containers, volumes, environment variables, etc.

Useful Commands

Create a Pod	View Pods	Describe a Pod	View Pod Logs	Delete a Pod
kubectl apply -f pod.yaml	kubectl get pods	kubectl describe pod nginx-pod	kubectl logs nginx-pod	kubectl delete pod nginx-pod

Keep Learning | Keep Building | Keep Growing

 Skills and Career Path in Software

A Pod is the smallest schedulable unit. Containers in a Pod share networking and can share volumes.

Key production interpretation

Lifecycle: Pending -> Running -> Succeeded/Failed; restart behavior depends on controller and restartPolicy.

Ephemeral identity: Pod IPs and pod instances change; Services provide stable discovery.

Logs: Use current and --previous logs for restarted containers.

Failure modes to recognize

- CrashLoopBackOff: container repeatedly starts and exits.
- OOMKilled: container exceeds memory limit.

- Pending: scheduling, image, storage, or quota problem.
- Readiness failure: pod runs but should not receive traffic.

SRE habit: Always separate Kubernetes object health from application health. A Pod can be Running while the service is still broken because readiness, credentials, dependencies, DNS, or downstream systems are failing.

4. Deployments & ReplicaSets

DevOps Series

Learn Practice Build Grow

Page 4/15



Kubernetes Basics

Deployments & ReplicaSets

Ensure High Availability for Your Applications!

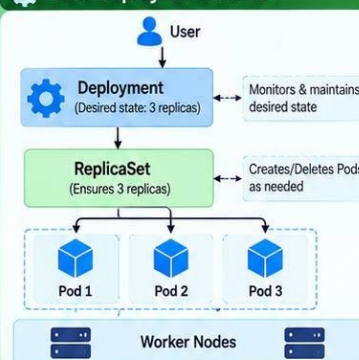
“Deployments help you run and manage stateless applications with automatic scaling and self-healing.”

What is a Deployment?

A Deployment is a higher-level Kubernetes object that manages ReplicaSets and provides declarative updates, scaling, rollbacks and self-healing for stateless applications.

- ✔ Maintains desired number of Pods
- ✔ Automatically replaces failed Pods
- ✔ Supports rolling updates and rollbacks
- ✔ Ideal for stateless applications

How Deployment Works?



```
graph TD
    User --> Deployment
    Deployment --> ReplicaSet
    ReplicaSet --> Pod1
    ReplicaSet --> Pod2
    ReplicaSet --> Pod3
    Pod1 --> WorkerNodes
    Pod2 --> WorkerNodes
    Pod3 --> WorkerNodes
```

Deployment Features

- Rolling Updates**
Update application without downtime.
- Rollbacks**
Easily revert to a previous version.
- Self-Healing**
Automatically replaces failed Pods.
- Scaling**
Scale up or down replicas manually or automatically.
- Revision History**
Keeps track of previous versions.
- Declarative Configuration**
Define desired state in YAML file.

What is a ReplicaSet?

A ReplicaSet (RS) ensures that a specified number of Pod replicas are always running. It is usually created and managed by a Deployment.

- ✔ Maintains the desired number of Pods
- ✔ Replaces failed or terminated Pods
- ✔ Uses label selectors to identify Pods
- ✔ Rarely created manually (handled by Deployment)

Example: Deployment YAML

```
apiVersion: apps/v1      # API version for Deployment
kind: Deployment          # Type of resource
metadata:
  name: nginx-deployment # Deployment name
  labels:
    app: nginx            # Labels
spec:
  replicas: 3             # Number of desired replicas
  selector:
    matchLabels:
      app: nginx          # Label selector
  template:
    metadata:
      labels:
        app: nginx        # Labels for Pods
    spec:
      containers:
        - name: nginx      # Container details
          image: nginx:latest # Docker image
          ports:
            - containerPort: 80 # Expose container port
```

Update a Deployment

- Update the image in YAML file
image: nginx:1.25
- Apply the changes
kubectl apply -f deployment.yaml
- Kubernetes performs a rolling update (replaces old Pods gradually)
- Verify the rollout
kubectl rollout status deployment/nginx-deployment

Rollback to Previous Version

- View rollout history
kubectl rollout history deployment/nginx-deployment
- Rollback to previous version
kubectl rollout undo deployment/nginx-deployment

Deployment vs ReplicaSet

Feature	Deployment	ReplicaSet
Purpose	Manages applications	Maintains Pod replicas
Created by	User	Created by Deployment
Supports updates	Yes (rolling updates, rollback)	No
Supports rollbacks	Yes	No
Scaling	Yes	Yes
Recommended usage	Use Deployment	Usually not created manually

Useful Commands

Create Deployment	Get Deployments	Describe Deployment	Scale Deployment	Delete Deployment
kubectl apply -f deployment.yaml	kubectl get deployments	kubectl describe deployment nginx-deployment	kubectl scale deployment nginx-deployment --replicas=5	kubectl delete deployment nginx-deployment

Keep Learning | Keep Building | Keep Growing

Skills and Career Path in Software

Deployments manage ReplicaSets, desired replicas, rollout strategy, and rollback history.

Key production interpretation

Desired state: Deployment says how many replicas and which pod template should exist.

RollingUpdate: Gradually replaces old pods; maxSurge/maxUnavailable govern availability.

Rollback: Rollout history can return to a known-good revision.

Failure modes to recognize

- Bad image/config causes rollout to stall.
- Readiness failures can make new pods unavailable.

- Insufficient capacity can prevent new replicas from scheduling.
- Selector/template mismatches can produce confusing ownership or traffic behavior.

SRE habit: Always separate Kubernetes object health from application health. A Pod can be Running while the service is still broken because readiness, credentials, dependencies, DNS, or downstream systems are failing.

5. Services

DevOps Series

Learn Practice Build Grow

Page 5/15



Kubernetes Basics

Services

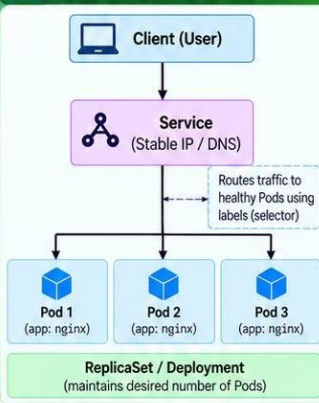
Expose and Access Your Applications!

What is a Service?

A Service is a Kubernetes object that provides a stable network endpoint to access a set of Pods. Since Pods are ephemeral and their IP addresses can change, a Service ensures consistent and reliable access to your application.

- ✓ Provides a stable IP/DNS name
- ✓ Load balances traffic across Pods
- ✓ Works even if Pods are recreated
- ✓ Supports different types (ClusterIP, NodePort, LoadBalancer, Headless)

How a Service Works?



```
graph TD
    Client[Client User] --> Service[Service Stable IP / DNS]
    Service -- "Routes traffic to healthy Pods using labels selector" --> Pod1[Pod 1 app: nginx]
    Service --> Pod2[Pod 2 app: nginx]
    Service --> Pod3[Pod 3 app: nginx]
    Pod1 & Pod2 & Pod3 --> RS[ReplicaSet / Deployment maintains desired number of Pods]
```

Types of Services

- ClusterIP**
 - Default type
 - Accessible only within the cluster
 - Used for internal communication
- NodePort**
 - Exposes the service on a static port on each node's IP
 - Accessible from outside using <NodeIP>:<NodePort>
- LoadBalancer**
 - Provisions an external load balancer (in cloud environments)
 - Provides a public IP to access the service
- Headless**
 - No cluster IP is assigned
 - Directly returns the IPs of Pods
 - Useful for StatefulSets or custom service discovery

Service YAML Example (ClusterIP)

```
apiVersion: v1 # API version
kind: Service # Type of resource
metadata:
  name: nginx-service # Service name
spec:
  type: ClusterIP # Service type
  selector: # Selects Pods with this label
    app: nginx
  ports:
    - protocol: TCP # Protocol
      port: 80 # Service port (inside cluster)
      targetPort: 80 # Container port
```

Accessing the Service

- 1 Get Service details
`kubectl get svc`
- 2 Describe the Service
`kubectl describe svc nginx-service`
- 3 Test from inside the cluster
`kubectl run test-pod --rm -it --image=busybox \`
`-- wget -qO- nginx-service`
- 4 Test from outside (NodePort/LoadBalancer)
`http://<NodeIP>:<NodePort>`
or
`http://<LoadBalancer-IP>`

Service Types Comparison

Type	Accessible From	Use Case
ClusterIP	Inside cluster only	Internal communication between services
NodePort	Outside (via Node IP + port)	Development and testing
LoadBalancer	Outside (via cloud LB)	Production applications
Headless	Inside cluster only	Service discovery (StatefulSets, custom needs)

Selector & Labels

- ✓ A Service uses label selectors to find and route traffic to matching Pods.
- ✓ Make sure the labels in the Service match the labels in your Pods (e.g., app: nginx).

Real-world Example

- ✓ Frontend Service (LoadBalancer) → Exposes UI to internet
- ✓ Backend Service (ClusterIP) → Internal communication
- ✓ Database Service (Headless) → For StatefulSet (e.g., MongoDB, PostgreSQL)

Key Takeaways

- ✓ Service provides stable access to Pods.
- ✓ Supports multiple types for different use cases.
- ✓ Uses label selectors to route traffic.
- ✓ Essential for building scalable applications.

"Pods may come and go, but a Service keeps your application reachable."

Keep Learning | Keep Building | Keep Growing

Skills and Career Path in Software

Services decouple stable access from ephemeral pod IPs by selecting pods and routing to endpoints.

Key production interpretation

ClusterIP: Internal virtual IP for service-to-service traffic.

NodePort: Exposes a port on every node; mostly useful for specific patterns/testing.

LoadBalancer: Cloud-integrated external load balancer.

Headless: No cluster IP; returns pod endpoints directly, common with StatefulSets.

Failure modes to recognize

- Service unreachable when selector does not match pod labels.

- Port/targetPort mismatch causes connection failures.
- No ready endpoints means Service has nowhere healthy to send traffic.
- NetworkPolicy can block traffic even when Service and endpoints look correct.

SRE habit: Always separate Kubernetes object health from application health. A Pod can be Running while the service is still broken because readiness, credentials, dependencies, DNS, or downstream systems are failing.

6. Namespaces & Labels

DevOps Series

Learn Practice Build Grow

Page 6/15



Kubernetes Basics

Namespaces & Labels

Organize and Manage Your Resources Efficiently!

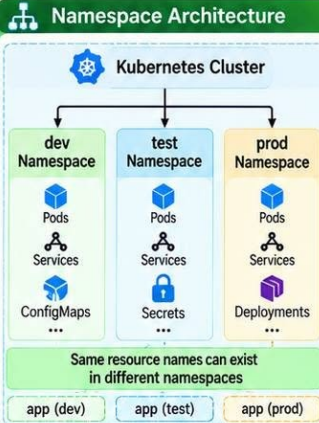
“Namespaces keep your resources organized, and Labels help you group, identify and manage them easily.”

What is a Namespace?

A **Namespace** is a logical separation within a Kubernetes cluster. It allows you to create isolated environments for different teams, applications or projects, so that resources with the same name can exist in different namespaces.

- ✓ Helps in organizing resources
- ✓ Provides isolation between teams environments
- ✓ Useful for multi-tenant clusters
- ✓ Default namespaces: default, kube-system, kube-public, kube-node-lease

Namespace Architecture



Same resource names can exist in different namespaces

app (dev) app (test) app (prod)

What are Labels?

Labels are key-value pairs that you can attach to Kubernetes resources. They help in organizing, selecting and managing resources.

- ✓ Used to identify and group resources
- ✓ Helpful in filtering and selecting resources
- ✓ Can be used with selectors (e.g., in Services, Deployments, etc.)
- ✓ You can define multiple labels for a resource

Label Example

app	=	nginx
env	=	dev
tier	=	frontend

key value

Useful Namespace Commands

- 1 List all namespaces
`kubectl get namespaces`
- 2 Create a namespace
`kubectl create namespace dev`
- 3 Delete a namespace
`kubectl delete namespace dev`
- 4 View resources in a namespace
`kubectl get pods -n dev`
- 5 Set default namespace (temporary)
`kubectl config set-context --current --namespace=dev`

Namespace YAML Example

```
apiVersion: v1
kind: Namespace
metadata:
  name: dev
  labels:
    env: development
    team: backend
```

This creates a namespace named "dev" with labels env=development and team=backend.

Label in a Pod YAML Example

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  namespace: dev
  labels:
    app: nginx
    env: dev
    tier: frontend
spec:
  containers:
    - name: nginx
      image: nginx:1.25
```

Assign namespace

Labels for this Pod

This Pod will be created in the "dev" namespace with the specified labels.

Working with Labels

- 1 Add a label to a resource
`kubectl label pod nginx-pod env=dev`
- 2 View labels of a resource
`kubectl get pod nginx-pod --show-labels`
- 3 Filter resources by label
`kubectl get pods -l app=nginx`
- 4 Update a label
`kubectl label pod nginx-pod env=prod --overwrite`
- 5 Remove a label
`kubectl label pod nginx-pod env-`

Common Use Cases

- ✓ Separate resources for different teams (dev, test, prod)
- ✓ Apply policies (resource quotas, RBAC) per namespace
- ✓ Use labels to manage and filter resources
- ✓ Deploy different versions of an application using labels
- ✓ Organize monitoring, logging and billing by namespace/labels

Best Practices

- ✓ Use meaningful namespace names (e.g., dev, test, prod)
- ✓ Use labels consistently (e.g., app, env, team)
- ✓ Combine labels with selectors in Services and Deployments
- ✓ Avoid too many namespaces (keep it simple)
- ✓ Use naming conventions for labels
- ✓ Clean up unused namespaces and resources

Pro Tip: Use namespaces and labels together to manage large and complex Kubernetes clusters effectively.

Keep Learning | Keep Building | Keep Growing

 Skills and Career Path in Software

Namespaces create logical boundaries; labels are the metadata glue used by Services, Deployments, policies, and observability.

Key production interpretation

Namespaces: Scope names and policies; not a security boundary by themselves.

Labels: Key/value tags used for selection and grouping.

Selectors: Power Service routing, ReplicaSet ownership, policies, and queries.

Failure modes to recognize

- Wrong namespace causes “not found” confusion.

- Label mismatch disconnects Service from pods.
- Broad selectors can unintentionally target too many workloads.
- Inconsistent labeling weakens monitoring and policy enforcement.

7. ConfigMaps & Secrets

DevOps Series

Learn Practice Build Grow

Page 7/15



Kubernetes Basics

ConfigMaps & Secrets

Manage Configuration and Sensitive Data Easily!

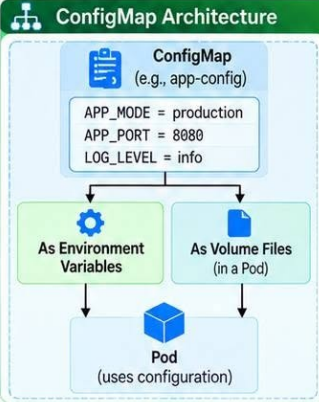
“ConfigMaps keep your configuration flexible, and Secrets keep your sensitive data safe in Kubernetes.”

What is a ConfigMap?

A **ConfigMap** is a Kubernetes object used to store non-sensitive configuration data in key-value pairs. It allows you to decouple configuration from your application code.

- ✓ Stores configuration as key-value pairs
- ✓ Can be used in Pods as environment variables or files
- ✓ Helps in managing different configurations for different environments (dev, test, prod)
- ✓ Not suitable for sensitive data (use Secrets for that)

ConfigMap Architecture



```
graph TD
    CM["ConfigMap  
(e.g., app-config)  
APP_MODE = production  
APP_PORT = 8080  
LOG_LEVEL = info"]
    CM --> EV["As Environment Variables"]
    CM --> VF["As Volume Files  
(in a Pod)"]
    EV --> P["Pod  
(uses configuration)"]
    VF --> P
```

ConfigMap YAML Example

```
apiVersion: v1 # Kubernetes API version
kind: ConfigMap # Resource type
metadata:
  name: app-config # ConfigMap name
  labels:
    app: myapp
data:
  # Key-value pairs
  APP_MODE: "production"
  APP_PORT: "8080"
  LOG_LEVEL: "info"
```

You can create a ConfigMap using literal values (as above) or from a file using:

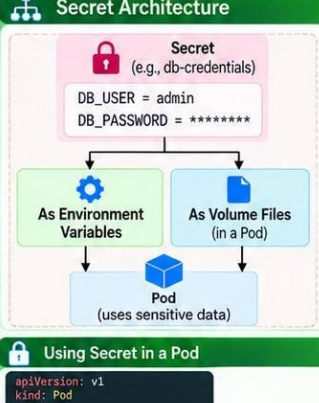
```
kubectl create configmap app-config --from-file=config.properties
```

What is a Secret?

A **Secret** is a Kubernetes object used to store sensitive information such as passwords, API keys, tokens, or certificates.

- ✓ Stores sensitive data in encrypted form (base64 encoded)
- ✓ Can be used as environment variables or mounted as files
- ✓ Helps keep sensitive data out of application code
- ✓ Access is restricted using RBAC

Secret Architecture



```
graph TD
    S["Secret  
(e.g., db-credentials)  
DB_USER = admin  
DB_PASSWORD = ****"]
    S --> EV["As Environment Variables"]
    S --> VF["As Volume Files  
(in a Pod)"]
    EV --> P["Pod  
(uses sensitive data)"]
    VF --> P
```

Secret YAML Example

```
apiVersion: v1 # Kubernetes API version
kind: Secret # Resource type
metadata:
  name: db-credentials # Secret name
  type: Opaque # Type (Opaque, tls, etc.)
data:
  # Base64 encoded values
  DB_USER: YWRtaW4= # echo -n 'admin' | base64
  DB_PASSWORD: bXlybXN1ZHMz # echo -n 'mypass123' | base64
```

To create a Secret from literal values:

```
kubectl create secret generic db-credentials \
  --from-literal=DB_USER=admin \
  --from-literal=DB_PASSWORD=mypass123
```

Using ConfigMap in a Pod

```
apiVersion: v1
kind: Pod
metadata:
  name: app-pod
spec:
  containers:
    - name: app
      image: nginx
      env:
        - name: APP_MODE
          valueFrom:
            configMapKeyRef:
              name: app-config
              key: APP_MODE
```

This exposes the **APP_MODE** key from the ConfigMap as an environment variable inside the Pod.

Using Secret in a Pod

```
apiVersion: v1
kind: Pod
metadata:
  name: app-pod
spec:
  containers:
    - name: app
      image: nginx
      env:
        - name: DB_USER
          valueFrom:
            secretKeyRef:
              name: db-credentials
              key: DB_USER
```

This exposes the **DB_USER** key from the Secret as an environment variable inside the Pod.

Best Practices

- ✓ Use ConfigMaps for non-sensitive configuration data.
- ✓ Use Secrets for sensitive information.
- ✓ Use meaningful names and labels.
- ✓ Separate configuration for different environments (dev, test, prod).
- ✓ Restrict access to Secrets using RBAC.
- ✓ Consider using external secret management tools (for example, HashiCorp Vault, AWS Secrets Manager).

Pro Tip: Secrets in Kubernetes are base64 encoded, not fully encrypted. Use additional tools for stronger security in production environments.

Keep Learning | Keep Building | Keep Growing

 Skills and Career Path in Software

Configuration should be externalized from images. ConfigMaps hold non-sensitive settings; Secrets hold sensitive data but require proper encryption and access controls.

Key production interpretation

Injection: Values can be exposed as environment variables or mounted files.

Update behavior: Mounted files may refresh; environment variables require pod restart/recreation.

Security: Base64 is encoding, not encryption. Use encryption at rest, RBAC, and external secret managers where appropriate.

Failure modes to recognize

- Missing secret/config key prevents container startup.
- Wrong value causes application errors despite healthy Kubernetes objects.
- Rotated credentials may require pod restart.
- Permissions can block a workload from reading a secret.

SRE habit: Always separate Kubernetes object health from application health. A Pod can be Running while the service is still broken because readiness, credentials, dependencies, DNS, or downstream systems are failing.

8. Volumes & Persistent Storage

DevOps Series

Learn Practice Build Grow

Page 8/15



Kubernetes Basics

Volumes & Persistent Storage

Store Your Data, Keep It Safe!

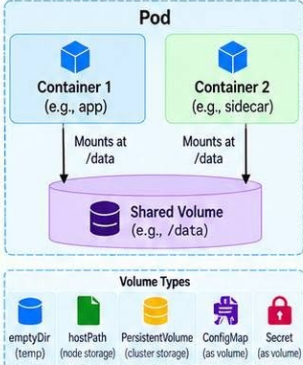
Containers are ephemeral, but your data doesn't have to be. Volumes and Persistent Storage keep your data safe and available."

Why Do We Need Volumes?

By default, data inside a container is ephemeral. If a Pod is deleted, the data is lost. Volumes allow you to store and persist data beyond the lifecycle of a container.

- ✓ Persist data beyond container lifecycle
- ✓ Share data between containers in the same Pod
- ✓ Store configuration files, logs, user uploads, etc.
- ✓ Use Persistent Volumes for long-term storage
- ✓ Essential for stateful applications (databases, message queues, etc.)

How Volumes Work?



Types of Volumes

Volume Type	Description
emptyDir	Temporary storage, deleted when Pod is removed
hostPath	Mounts a directory from the node's filesystem
PersistentVolume (PV)	Cluster-wide storage resource
PersistentVolumeClaim (PVC)	Request for storage by a user (application)
ConfigMap	Use a ConfigMap as a volume to mount configuration files
Secret	Use a Secret as a volume to mount sensitive data

Example: emptyDir Volume

```
apiVersion: v1
kind: Pod
metadata:
  name: emptydir-pod
spec:
  containers:
    - name: app
      image: nginx
      volumeMounts:
        - name: cache-volume
          mountPath: /cache
      volumes:
        - name: cache-volume
          emptyDir: {}
```

Creates a temporary volume. Data is lost when the Pod is deleted.

Example: hostPath Volume

```
apiVersion: v1
kind: Pod
metadata:
  name: hostpath-pod
spec:
  containers:
    - name: app
      image: nginx
      volumeMounts:
        - name: data-volume
          mountPath: /data
      volumes:
        - name: data-volume
          hostPath:
            path: /var/log
            type: Directory
```

Mounts a directory from the node's filesystem into the Pod.

Example: PersistentVolume & PVC

```
PersistentVolume (PV)
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-demo
spec:
  capacity:
    storage: 1Gi
  accessModes: ["ReadWriteOnce"]
  hostPath:
    path: /mnt/data

PersistentVolumeClaim (PVC)
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-demo
spec:
  accessModes: ["ReadWriteOnce"]
  resources:
    requests:
      storage: 1Gi
```

Defines a storage resource in the cluster (e.g., 1 Gi storage).

Requests storage from the cluster using the defined PV.

Common Use Cases

- Database storage (MySQL, PostgreSQL)
- Application logs (e.g., /var/log)
- User uploaded files
- Configuration files (via ConfigMap)
- Certificates and keys (via Secret)

Useful Commands

List Persistent Volumes	<code>kubectl get pv</code>
List Persistent Volume Claims	<code>kubectl get pvc</code>
Describe a PV	<code>kubectl describe pv <name></code>
Describe a PVC	<code>kubectl describe pvc <name></code>
Check volume mounts in a Pod	<code>kubectl describe pod <name></code>

Best Practices

- ✓ Use Persistent Volumes for important data.
- ✓ Use appropriate storage classes (cloud or on-prem).
- ✓ Use PVCs instead of directly using PVs.
- ✓ Set correct access modes (RWO, RWX, ROX).
- ✓ Use ConfigMaps and Secrets for configuration files.
- ✓ Regularly backup important data.
- ✓ Monitor storage usage.

Pro Tip: In cloud environments, use managed storage (Alibaba Cloud, AWS EBS, Azure Disk, etc.) instead of hostPath.

Keep Learning | Keep Building | Keep Growing

Skills and Career Path in Software

Container filesystems are ephemeral. Volumes provide shared or durable storage depending on the type.

Key production interpretation

emptyDir: Lives for the life of the Pod; useful for scratch space and sidecar sharing.

hostPath: Node-local path; risky and not portable for most production workloads.

PV/PVC: Separates storage supply from workload request.

StorageClass: Defines dynamic provisioning behavior.

Failure modes to recognize

- PVC Pending blocks workloads waiting for storage.

- Mount errors can keep containers in ContainerCreating.
- Node-local storage can create hidden coupling.
- Full disks or volumes can cause application failure even if pods stay Running.

SRE habit: Always separate Kubernetes object health from application health. A Pod can be Running while the service is still broken because readiness, credentials, dependencies, DNS, or downstream systems are failing.

9. kubectl Commands

DevOps Series

Learn Practice Build Grow

Page 9/15



Kubernetes Basics

kubectl Commands

Manage Your Cluster with kubectl!

“kubectl is your control center to communicate with the Kubernetes cluster. Learn a few commands and you can manage everything!”

What is kubectl?

kubectl is a command-line tool used to interact with a Kubernetes cluster. It allows you to deploy applications, inspect and manage resources, view logs, and perform many other tasks.

- Communicates with the Kubernetes API server
- Helps you manage cluster resources
- Can be used locally or from a jump server
- Supports imperative commands and YAML files
- Essential tool for DevOps and Kubernetes administration

Basic Command Structure

`kubectl [command] [type] [name] [options]`

- The action you want to perform (get, create, apply, etc.)
- The type of resource (pod, svc, deployment, etc.)
- The name of the resource (optional)
- Additional flags and options (optional)

Example:

```
kubectl get pods -n dev -o wide
```

Common Resource Types (Short Names)

Resource Type	Short Name	Resource Type	Short Name
Pod	po	ConfigMap	cm
Deployment	deploy	Secret	secret
ReplicaSet	rs	Ingress	ing
Service	svc	DaemonSet	ds
Namespace	ns	StatefulSet	sts
Node	no	Job	job
PersistentVolume	pvc	CronJob	cj
PersistentVolumeClaim	pvc	HorizontalPodAutoscaler	hpa

Frequently Used kubectl Commands

Task / Description	Command
View cluster information	<code>kubectl cluster-info</code>
View nodes	<code>kubectl get nodes</code>
List all resources	<code>kubectl get all</code>
List resources in a namespace	<code>kubectl get all -n <namespace></code>
Create a resource from YAML	<code>kubectl apply -f <file.yaml></code>
View resource details	<code>kubectl describe <type> <name></code>
View logs of a Pod	<code>kubectl logs <pod-name></code>
Follow logs (real-time)	<code>kubectl logs -f <pod-name></code>
Execute a command in a Pod	<code>kubectl exec -it <pod-name> -- /bin/bash</code>
Delete a resource	<code>kubectl delete <type> <name></code>

Filtering & Output Options

Option	Description
<code>-n <namespace></code>	Specify namespace
<code>-o wide</code>	Show more details
<code>-o yaml</code>	Output in YAML format
<code>-o json</code>	Output in JSON format
<code>--all-namespaces</code>	Show resources in all namespaces
<code>-l <label></code>	Filter using labels (e.g., <code>-l app=nginx</code>)
<code>--sort-by=<field></code>	Sort output (e.g., <code>--sort-by=.metadata.name</code>)
<code>-w</code>	Watch for changes (real-time)

Managing Applications

Task	Command
Create a deployment	<code>kubectl create deployment <name> --image=<image></code>
Scale a deployment	<code>kubectl scale deployment <name> --replicas=3</code>
Update image	<code>kubectl set image deployment <name> <container>=<image></code>
View rollout status	<code>kubectl rollout status deployment <name></code>
View rollout history	<code>kubectl rollout history deployment <name></code>
Undo a rollout	<code>kubectl rollout undo deployment <name></code>
Expose a deployment	<code>kubectl expose deployment <name> --type=NodePort --port=80</code>

Practical Examples

- Deploy an Nginx application
`kubectl create deployment nginx --image=nginx`
- Expose it as a service
`kubectl expose deployment nginx --type=NodePort --port=80`
- Check running resources
`kubectl get all`
- View logs
`kubectl logs -f deployment/nginx`
- Delete the application
`kubectl delete deployment nginx`

Tips & Best Practices

- Use tab completion to speed up commands.
- Always specify the namespace when working in non-default environments.
- Use descriptive names for resources.
- Combine kubectl with YAML files for repeatable deployments.
- Use labels to filter and organize resources.
- Regularly check logs, events and describe output for troubleshooting.

Useful Aliases (Optional)

You can create aliases to save time:

```
alias k=kubectl
alias kg='kubectl get'
alias kd='kubectl describe'
alias kl='kubectl logs'
alias ka='kubectl apply -f'
alias kn='kubectl config set-context --current --namespace'
```

Pro Tip: Use `kubectl --help` to explore more commands and options.

Keep Learning | Keep Building | Keep Growing

 Skills and Career Path in Software

kubectl is the primary investigation tool for object state, events, logs, exec, rollouts, and resource inspection.

Key production interpretation

Get: List and summarize resources.

Describe: See conditions, events, scheduling, mounts, and container state.

Logs: Inspect application output; `--previous` is critical for crash loops.

Exec: Run targeted checks from inside a pod.

Rollout: Watch deployments and rollback if necessary.

Failure modes to recognize

- Avoid making destructive changes before capturing evidence.
- Use `-n` explicitly to prevent namespace mistakes.
- Prefer `describe/events` before guessing.
- Use label selectors and output formats for precise filtering.

10. YAML Manifests

DevOps Series

Learn Practice Build Grow

Page 10/15



Kubernetes Basics

YAML Manifests

Define, Deploy and Manage Your Applications!

“YAML is how you tell Kubernetes what to do. Write a manifest once, and you can create the same environment anywhere!”

What is a YAML Manifest?

YAML (YAML Ain't Markup Language) is a human-readable format used to define Kubernetes resources. You describe the desired state of your application (like Pods, Deployments, Services, etc.) in a YAML file, and Kubernetes makes it happen.

- ✓ Defines desired state of resources
- ✓ Easy to read and write
- ✓ Can be version controlled (Git)
- ✓ Used with `kubectl apply -f <file.yaml>`
- ✓ Supports all Kubernetes objects (Pod, Deployment, Service, etc.)

Basic YAML Structure

```
# Kubernetes API version
apiVersion: v1
# Type of resource (Pod, Deployment, etc.)
kind: Pod
# Metadata about the resource
metadata:
  name: my-app
  labels:
    app: my-app
# Specification of the resource
spec:
  containers:
    - name: my-container
      image: nginx
```

apiVersion	API version of Kubernetes
kind	Type of resource
metadata	Name, labels and other info
spec	Desired state / configuration

Pod YAML Example

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  labels:
    app: nginx
spec:
  containers:
    - name: nginx
      image: nginx:1.25
      ports:
        - containerPort: 80
```

This creates a simple Pod running an Nginx container on port 80.

Deployment YAML Example

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
          ports:
            - containerPort: 80
```

This creates a Deployment with 3 replicas of an Nginx Pod.

Service YAML Example

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  type: NodePort
  selector:
    app: nginx
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080
```

This exposes the Nginx Deployment using a NodePort service on port 30080.

ConfigMap & Secret Examples

ConfigMap (Store non-sensitive data)

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_MODE: "production"
  APP_NAME: "myapp"
```

Store configuration data as key-value pairs.

Secret (Store sensitive data)

```
apiVersion: v1
kind: Secret
metadata:
  name: db-credentials
type: Opaque
data:
  DB_USER: YWRtaW4=
  DB_PASSWORD: bXlwYXNzMTIz
```

Store sensitive data (base64 encoded).

Apply and Manage Manifests

- 1 Apply a manifest `kubectl apply -f file.yaml`
- 2 View resources `kubectl get all`
- 3 Describe a resource `kubectl describe <resource> <name>`
- 4 Delete a resource `kubectl delete -f file.yaml`

Best Practices

- ✓ Use meaningful names and labels
- ✓ Use separate YAML files for each resource
- ✓ Keep manifests in version control (Git)
- ✓ Use ConfigMaps and Secrets for configuration
- ✓ Use namespaces to organize resources
- ✓ Validate your YAML files (`kubectl apply --dry-run=client -f file.yaml`)
- ✓ Follow consistent indentation and formatting

Common File Organization

```
k8s-manifests/
├── deployment.yaml # Deployment resource
├── service.yaml    # Service resource
├── configmap.yaml  # ConfigMap
├── secret.yaml     # Secret
└── namespace.yaml # Namespace (optional)
```

Pro Tip: Keep your manifests organized in folders, especially for larger projects.

Keep Learning | Keep Building | Keep Growing

Skills and Career Path in Software

YAML describes desired Kubernetes state. Small manifest mistakes can create large production effects.

Key production interpretation

apiVersion/kind: Select the API and object type.

metadata: Names, namespaces, labels, annotations.

spec: Desired state such as image, replicas, ports, resources, probes, volumes.

Version control: Manifests in Git make changes reviewable, repeatable, and auditable.

Failure modes to recognize

- Invalid YAML/API fields fail admission.

- Valid YAML with wrong semantics can deploy successfully but behave incorrectly.
- Incorrect selectors/ports/probes are common outage triggers.
- Diff changes before apply and use policy checks where possible.

11. Networking & Services

DevOps Series

Learn Practice Build Grow

Page 11/15



Kubernetes Basics

Networking & Services

Connect Your Applications Inside and Outside the Cluster!

“Services give a stable network identity to your Pods, and Ingress helps you expose your applications to the outside world.”

Why Do We Need Services?

Pods are ephemeral. Their IP addresses change when they are recreated. Services provide a stable network endpoint (IP/DNS) to access Pods, even if the Pods change.

- Provides a stable IP and DNS name
- Load balances traffic across multiple Pods
- Decouples clients from Pods
- Supports different access types (ClusterIP, NodePort, LoadBalancer, etc.)
- Essential for microservices communication

Service Architecture

Service routes traffic to healthy Pods using label selectors

Types of Services

Type	Description	Use Case
ClusterIP	Exposes the service on a cluster-internal IP (default).	Used for internal communication between services.
NodePort	Exposes the service on a static port on every node.	Access from outside the cluster (dev/test).
LoadBalancer	Creates an external load balancer (cloud provider).	Expose your app to the internet (production).
ExternalName	Maps the service to an external DNS name.	Access external services (e.g., database outside cluster).

ClusterIP Service YAML Example

```
apiVersion: v1
kind: Service
metadata:
  name: myapp-service
spec:
  type: ClusterIP
  selector:
    app: myapp
  ports:
    - port: 80
      targetPort: 80
```

Creates an internal service accessible only within the cluster.

NodePort Service YAML Example

```
apiVersion: v1
kind: Service
metadata:
  name: myapp-nodeport
spec:
  type: NodePort
  selector:
    app: myapp
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080
```

Exposes the service on a static port (30080) on every node.

LoadBalancer Service YAML Example

```
apiVersion: v1
kind: Service
metadata:
  name: myapp-lb
spec:
  type: LoadBalancer
  selector:
    app: myapp
  ports:
    - port: 80
      targetPort: 80
```

Creates an external load balancer (works on cloud providers like AWS, Azure, GCP).

Ingress Overview

Ingress manages external access to services using HTTP/HTTPS. It provides routing rules, SSL/TLS termination, and host-based routing.

- Routes traffic based on domain name or path
- Can handle multiple services
- Supports SSL/TLS (HTTPS)
- Requires an Ingress Controller (e.g., Nginx Ingress)
- Simplifies external access compared to NodePort/LB

Ingress YAML Example

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: myapp-ingress
spec:
  rules:
    - host: myapp.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: myapp-service
                port:
                  number: 80
```

Routes traffic from myapp.example.com to the myapp-service on port 80.

Useful Commands

- List all services: `kubectl get svc`
- Describe a service: `kubectl describe svc <name>`
- List ingress resources: `kubectl get ingress`
- Describe an ingress: `kubectl describe ingress <name>`

Best Practices

- Use ClusterIP for internal communication.
- Use Ingress for HTTP/HTTPS applications.
- Use meaningful names and labels.
- Configure TLS for secure access.
- Monitor service and ingress traffic.

Keep Learning | Keep Building | Keep Growing

Skills and Career Path in Software

A successful request may cross DNS, Service VIP, endpoint selection, pod network, NetworkPolicy, Ingress, and external firewalls.

Key production interpretation

Service discovery: DNS names resolve Services or external names.

Endpoint selection: Ready pods become service endpoints.

Ingress: HTTP/HTTPS routing into Services.

Policies: NetworkPolicy controls allowed pod traffic when enforced by the CNI.

Failure modes to recognize

- DNS resolution failure.
- Connection refused when destination is reachable but nothing listens.
- Timeout when routing/firewall/policy blocks or path is black-holed.
- Ingress 502/503 when backend is unhealthy, unavailable, or misconfigured.

SRE habit: Always separate Kubernetes object health from application health. A Pod can be Running while the service is still broken because readiness, credentials, dependencies, DNS, or downstream systems are failing.

12. Deployments & Scaling

DevOps Series

Learn Practice Build Grow

Page 12/15



Kubernetes Basics

Deployments & Scaling

Run, Update and Scale Your Applications with Ease!

“Deployments give you power to run, update and scale your applications without downtime. Kubernetes handles the rest!”

What is a Deployment?

A **Deployment** is a Kubernetes object used to run and manage stateless applications. It maintains the desired number of Pod replicas, handles updates (rolling updates), and provides self-healing if a Pod fails.

- ✓ Manages ReplicaSets and Pods
- ✓ Ensures desired number of replicas are running
- ✓ Supports rolling updates and rollbacks
- ✓ Self-heals by replacing failed Pods
- ✓ Widely used for stateless applications (e.g., web apps, APIs)

Deployment Architecture

- Manages ReplicaSets
- Maintains desired replicas
- Handles rolling updates
- Self-healing

If a Pod fails, Kubernetes automatically creates a new one to match the desired state.

Deployment YAML Example

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp-deployment
  labels:
    app: myapp
spec:
  replicas: 3
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: myapp
          image: nginx:1.25
          ports:
            - containerPort: 80
```

Annotations in the YAML: Deployment name, Number of pod replicas, Label selector to identify Pods, Pod template (specification), Container details (image & port)

Scaling a Deployment

You can scale a Deployment to increase or decrease the number of Pod replicas based on your application's needs.

```
# Scale to 5 replicas
kubectl scale deployment myapp-deployment --replicas=5

# Check the status
kubectl get pods

# Check deployment details
kubectl get deployment myapp-deployment
```

Rolling Updates

Deployments support rolling updates, which allow you to update your application with zero downtime. Kubernetes gradually replaces old Pods with new ones.

```
# Update image to a new version
kubectl set image deployment/myapp-deployment myapp=nginx:1.26

# Check rollout status
kubectl rollout status deployment/myapp-deployment

# View rollout history
kubectl rollout history deployment/myapp-deployment

# Rollback to previous version
kubectl rollout undo deployment/myapp-deployment
```

Update Strategies

Strategy	Description	Use Case
RollingUpdate (Default)	Gradually replaces old Pods with new ones.	Most common (safe & zero downtime)
Recreate	Terminates all old Pods before creating new ones.	Simple applications (with downtime)

You can also configure advanced settings like `maxUnavailable` and `maxSurge` for rolling updates.

Useful Commands

Task	Command
Create a deployment	<code>kubectl apply -f deployment.yaml</code>
Get deployments	<code>kubectl get deployments</code>
Describe a deployment	<code>kubectl describe deployment <name></code>
Scale a deployment	<code>kubectl scale deployment <name> --replicas=5</code>
Update image	<code>kubectl set image deployment/<name> <container>=<image></code>
Check rollout status	<code>kubectl rollout status deployment/<name></code>
View rollout history	<code>kubectl rollout history deployment/<name></code>
Rollback deployment	<code>kubectl rollout undo deployment/<name></code>
Delete deployment	<code>kubectl delete deployment <name></code>

Real-world Use Case

- ✓ Deploy a web application (e.g., Nginx)
- ✓ Scale based on traffic (manual or auto)
- ✓ Update to a new version without downtime
- ✓ Rollback if something goes wrong
- ✓ Keep the application always available

Pro Tip: Use `kubectl rollout status` to monitor updates and ensure your application is running smoothly.

Best Practices

- ✓ Use meaningful names and labels.
- ✓ Set resource requests and limits.
- ✓ Use rolling updates for zero downtime.
- ✓ Configure readiness and liveness probes.
- ✓ Keep previous versions for quick rollback.
- ✓ Use Horizontal Pod Autoscaler (HPA) for automatic scaling (next topic).
- ✓ Monitor your deployments (e.g., with Prometheus + Grafana).

Keep Learning | Keep Building | Keep Growing

 Skills and Career Path in Software

Scaling and rollout behavior directly affects availability, capacity, and recovery time.

Key production interpretation

Manual scaling: Changes replica count directly.

Autoscaling: HPA adjusts replicas from metrics; cluster autoscaling adds/removes node capacity.

Readiness: Only ready pods should receive traffic.

Rollout strategy: Controls how much capacity can be unavailable during changes.

Failure modes to recognize

- Too few replicas increases blast radius.

- Aggressive limits cause OOM/throttling.
- HPA without capacity can leave pods Pending.
- A rollout with failing readiness can stall or reduce available capacity.

SRE habit: Always separate Kubernetes object health from application health. A Pod can be Running while the service is still broken because readiness, credentials, dependencies, DNS, or downstream systems are failing.

13. ConfigMaps & Secrets - Production Practices

DevOps Series

Learn Practice Build Grow

Page 13/15



Kubernetes Basics

ConfigMaps & Secrets

Externalize Configuration, Keep Sensitive Data Safe!

“ConfigMaps keep your configuration separate from your code, and Secrets keep your sensitive data safe in Kubernetes.”

Why ConfigMaps & Secrets?

Applications have configuration data and sensitive data (like passwords, API keys, etc.). Keeping them inside images is not secure or flexible. Kubernetes provides ConfigMaps and Secrets to manage this data separately.

- ✓ Keep configuration outside the image
- ✓ Easily update configuration
- ✓ Manage sensitive data securely
- ✓ Use the same config/secret across multiple Pods
- ✓ Supports environment variables and mounted files

How They Work?

ConfigMaps and Secrets can be used as environment variables or mounted as files inside Pods.

Key Differences

Feature	ConfigMap	Secret
Purpose	Stores non-sensitive configuration data	Stores sensitive data (passwords, keys, certificates)
Data Format	Plain text (key-value)	Base64 encoded (key-value)
Use Cases	App configuration, feature flags, URLs, etc.	Database passwords, API keys, TLS certs, etc.
Security	Not encrypted (by default)	Base64 encoded (can be encrypted at rest)
Example	app.properties, config.yaml	db-password, api-key

ConfigMap Example

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_ENV: "production"
  APP_NAME: "myapp"
  APP_URL: "https://myapp.com"
```

Creates a ConfigMap with key-value pairs.

Secret Example

```
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
data:
  DB_USER: YWRtaW4=
  DB_PASSWORD: bXlQdHszMTIz
```

Creates a Secret. The data is Base64 encoded.

Using in a Pod (Environment Variables)

```
apiVersion: v1
kind: Pod
metadata:
  name: app-pod
spec:
  containers:
    - name: app
      image: myapp:latest
      env:
        - name: APP_ENV
          valueFrom:
            configMapKeyRef:
              name: app-config
              key: APP_ENV
        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef:
              name: db-secret
              key: DB_PASSWORD
```

Uses ConfigMap and Secret as environment variables.

Using in a Pod (Mounted as Volume)

```
apiVersion: v1
kind: Pod
metadata:
  name: app-pod-volume
spec:
  containers:
    - name: app
      image: myapp:latest
      volumeMounts:
        - name: config-vol
          mountPath: /etc/config
        - name: secret-vol
          mountPath: /etc/secret
      volumes:
        - name: config-vol
          configMap:
            name: app-config
        - name: secret-vol
          secret:
            secretName: db-secret
```

Mounts ConfigMap and Secret as files inside the Pod.

Useful Commands

Task	Command
Create ConfigMap (from literals)	kubectl create configmap app-config --from-literal=APP_ENV=prod
Create ConfigMap (from file)	kubectl create configmap app-config --from-file=config.yaml
Create Secret (from literals)	kubectl create secret generic db-secret --from-literal=DB_USER=admin --from-literal=DB_PASSWORD=myPass123
Create Secret (from file)	kubectl create secret generic tls-secret --from-file=tls.crt --from-file=tls.key
Get ConfigMaps	kubectl get configmaps
Get Secrets	kubectl get secrets
Describe	kubectl describe configmap <name> # or kubectl describe secret <name>

Best Practices

- ✓ Use ConfigMaps for non-sensitive data only.
- ✓ Use Secrets for sensitive data.
- ✓ Avoid storing sensitive data in plain text (use Base64 or encryption).
- ✓ Use RBAC to restrict access to Secrets.
- ✓ Mount as files for larger configuration data.
- ✓ Use environment variables for simple key-value pairs.
- ✓ Keep configuration and secrets in separate files.
- ✓ Rotate secrets regularly.

Pro Tip: For extra security, use tools like Sealed Secrets, HashiCorp Vault, or cloud provider secret managers (AWS Secrets Manager, Azure Key Vault, etc.).

Keep Learning | Keep Building | Keep Growing

 Skills and Career Path in Software

Production reliability depends on safe config and secret lifecycle management.

Key production interpretation

Separation: Keep config and secrets separate from images and source code.

Least privilege: Restrict secret access via service accounts and RBAC.

External management: AWS Secrets Manager, Vault, or similar systems can centralize rotation and audit.

Change safety: Coordinate secret rotations with consumers and restart behavior.

Failure modes to recognize

- Expired credentials produce auth errors.

- Malformed secret payloads break apps at startup or runtime.
- Secret name/key drift between environments causes deployment failures.
- Rotation without compatibility can create outages.

SRE habit: Always separate Kubernetes object health from application health. A Pod can be Running while the service is still broken because readiness, credentials, dependencies, DNS, or downstream systems are failing.

14. Volumes & Persistent Storage - Production Practices

DevOps Series

Learn Practice Build Grow

Page 14/15



Kubernetes Basics

Volumes & Persistent Storage

Keep Your Data Safe, Even If Pods Die!

“Containers are ephemeral, but your data doesn’t have to be. Kubernetes Volumes and Persistent Volumes help you keep your data safe and available!”

Why Do We Need Storage?

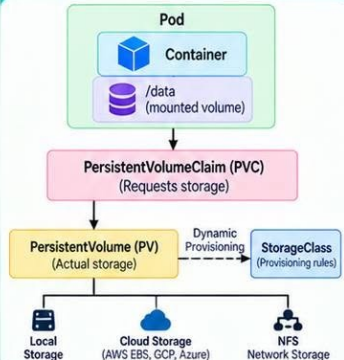
Containers are ephemeral, which means data inside a container can be lost when the Pod is deleted. Kubernetes provides Volumes and Persistent Volumes (PV) to store data outside the container.

- ✓ Persists data beyond Pod lifetime
- ✓ Supports different storage backends (local, cloud, network, etc.)
- ✓ Can be shared across multiple Pods
- ✓ Used for stateful applications (databases, file storage, etc.)
- ✓ Provides dynamic and static provisioning options

Types of Storage in Kubernetes

EmptyDir	Temporary storage, exists as long as the Pod runs.
HostPath	Mounts a directory from the node's filesystem. (Not recommended for production)
PersistentVolume (PV)	Cluster-wide storage resource.
PersistentVolumeClaim (PVC)	A request for storage from a PV.
StorageClass	Defines how storage is provisioned (dynamic).

How It Works?



1. EmptyDir Volume Example

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-emptydir
spec:
  containers:
    - name: nginx
      image: nginx
      volumeMounts:
        - name: cache
          mountPath: /usr/share/nginx/html
      volumes:
        - name: cache
          emptyDir: {}
```

Data is stored temporarily and is deleted when the Pod is removed.

2. PersistentVolume (PV) Example

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-demo
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  hostPath:
    path: /mnt/data
```

Defines a piece of storage in the cluster (can be local or cloud storage).

3. PersistentVolumeClaim (PVC) Example

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-demo
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: standard
```

Requests storage from a matching PersistentVolume (PV).

Common Storage Options

Type	Description	Use Case
emptyDir	Temporary storage for a Pod	Caching, temp files
hostPath	Mounts node's filesystem	Development / Testing
NFS	Network file system	Shared storage
Cloud Storage (AWS EBS, GCP, Azure)	Persistent block storage	Production workloads
StorageClass (dynamic)	Automatically provisions storage	Scalable environments

Useful Commands

- `kubectl get pv` List all PersistentVolumes
- `kubectl get pvc` List all PersistentVolumeClaims
- `kubectl describe pv <name>` View PV details
- `kubectl describe pvc <name>` View PVC details
- `kubectl get storageclass` List StorageClasses

Best Practices

- ✓ Use PersistentVolumes for important data.
- ✓ Use StorageClasses for dynamic provisioning.
- ✓ Choose the right access mode (RWO, RWX, etc.).
- ✓ Set appropriate storage limits.
- ✓ Use cloud storage for production workloads.
- ✓ Understand the reclaim policy (Retain, Delete, Recycle).
- ✓ Regularly backup important data.

Pro Tip: Use StatefulSets for stateful applications like databases to manage persistent storage easily.

Keep Learning | Keep Building | Keep Growing

 Skills and Career Path in Software

Stateful workloads require careful storage lifecycle, backup, capacity, and failure-domain planning.

Key production interpretation

Access modes: RWO/RWX/ROX affect which workloads can mount a volume.

Reclaim policy: Retain/Delete affects data after PVC/PV lifecycle events.

Backups: Persistent is not the same as backed up.

Failure domains: Storage zone/region constraints can affect scheduling and recovery.

Failure modes to recognize

- PVC Pending from StorageClass or topology mismatch.

- Attach/mount errors after node failure.
- Volume exhaustion or inode exhaustion.
- Data loss risk from wrong reclaim or backup assumptions.

SRE habit: Always separate Kubernetes object health from application health. A Pod can be Running while the service is still broken because readiness, credentials, dependencies, DNS, or downstream systems are failing.

15. Recap & Next Steps

DevOps Series

Learn Practice Build Grow

Page 15/15



Kubernetes Basics Recap & Next Steps

Key Takeaways, Real-World Impact and What's Next!

“Kubernetes is not just a tool, it's a foundation for the future of scalable, reliable and cloud-native applications.”

What We Covered?

- 1 What is Kubernetes?
- 2 Architecture (Master & Worker Nodes)
- 3 Pods, Labels and Selectors
- 4 Deployments & ReplicaSets
- 5 Services (ClusterIP, NodePort, LoadBalancer)
- 6 YAML Manifests
- 7 Networking & Ingress
- 8 ConfigMaps & Secrets
- 9 Volumes & Persistent Storage
- 10 Deployments & Scaling
- 11 Networking & Services (Advanced)
- 12 ConfigMaps & Secrets (Advanced)
- 13 Volumes & Persistent Storage (Advanced)
- 14 Real-world Use Cases
- 15 Recap & Next Steps

Key Benefits of Kubernetes

- ✓ **Scalability** – Easily scale applications up or down.
- ✓ **High Availability** – Automatic recovery from failures.
- ✓ **Self-Healing** – Replaces unhealthy containers.
- ✓ **Portability** – Runs on any cloud or on-premises.
- ✓ **Flexibility** – Supports many workloads and languages.
- ✓ **Efficiency** – Better resource utilization.
- ✓ **Ecosystem** – Large community and tool support.

Real-World Impact

- Used by top companies like **Google, Amazon, Microsoft, Netflix, Spotify** and many more.
- Powers millions of applications worldwide.
- Essential skill for DevOps, SRE and Cloud Engineers.
- Helps organizations deliver faster, more reliable and scalable software.

Kubernetes in the DevOps Workflow

Code → Build (Image) → Push to Registry → Deploy to Kubernetes → Monitor & Scale

Kubernetes brings everything together – from build to production – with automation and scalability!

Next Steps in Your Learning Journey

- ✓ Practice with real-world projects (e.g., deploy a full-stack application).
- ✓ Explore Helm (package manager for Kubernetes).
- ✓ Learn GitOps (ArgoCD / Flux).
- ✓ Dive into monitoring (Prometheus + Grafana).
- ✓ Understand security (RBAC, Network Policies).
- ✓ Explore service mesh (Istio).
- ✓ Work on multi-cluster and production setups.
- ✓ Get hands-on with cloud providers (EKS, GKE, AKS).
- ✓ Keep learning and stay updated with new features!

Quick Tips for Success

- ✓ Practice regularly with hands-on labs.
- ✓ Use meaningful names and labels.
- ✓ Keep your YAML files organized.
- ✓ Understand the “why” behind each concept.
- ✓ Start with small projects and gradually work on larger ones.
- ✓ Follow best practices and official documentation.
- ✓ Be part of the Kubernetes community.
- ✓ Never stop learning – the ecosystem keeps evolving!

Recommended Resources

-  **Kubernetes Official Documentation**
<https://kubernetes.io/docs/>
-  **Kubernetes by Example**
<https://kubernetesbyexample.com/>
-  **Play with Kubernetes**
<https://labs.play-with-k8s.com/>
-  **CNCF Learning Paths**
<https://www.cncf.io/>
-  **KillerCoda / Katacoda Scenarios**
<https://killercode.com/>

Final Message

“You’ve completed the Kubernetes Basics series! 🎉 Now it’s time to practice, build real projects and take your DevOps skills to the next level. Remember – learning never stops, and every expert was once a beginner. Keep building, keep exploring and keep growing!”

Next Stop... Real-World Projects! 🚀

Keep Learning | Keep Building | Keep Growing

 **Skills and Career Path in Software**

Kubernetes reliability comes from combining platform knowledge with disciplined operations.

Key production interpretation

Observe: Metrics, logs, events, traces, and health checks.

Respond: Mitigate impact, communicate status, and recover service.

Learn: RCA/postmortem and corrective actions.

Prevent: Better probes, limits, tests, policies, automation, and runbooks.

Failure modes to recognize

- Practice troubleshooting in a non-production lab.

- Learn observability tooling such as Prometheus/Grafana and centralized logs.
- Understand SLOs, error budgets, incident roles, and postmortems.
- Treat every incident as a chance to reduce future toil and risk.

Production Troubleshooting Cheat Sheet



Kubernetes Production Troubleshooting Cheat Sheet



10 common issues • symptoms • first checks • useful kubectl commands

1 CrashLoopBackOff

Symptoms

- Pod repeatedly restarts.

First checks

- Logs --previous
- Describe pod
- Inspect config/env
- Check probes

```
> kubectl logs <pod> --previous
```

2 ImagePullBackOff

Symptoms

- Image cannot be pulled.

First checks

- Image/tag
- Registry access
- ImagePullSecrets

```
> kubectl describe pod <pod>
```

3 Pending

Symptoms

- Pod not scheduled.

First checks

- CPU/memory
- Node availability
- Taints/tolerations
- Affinity
- PVC

```
> kubectl describe pod <pod>
```

4 Service Unreachable

Symptoms

- Connection timeout / refused.

First checks

- Endpoints
- Pod labels/selectors
- Service type
- NetworkPolicy

```
> kubectl get endpoints <svc>
```

5 Ingress 502/503

Symptoms

- Gateway errors.

First checks

- Backend health
- Service port
- Ingress rules
- Readiness
- Controller logs

```
> kubectl describe ingress <name>
```

6 OOMKilled

Symptoms

- Container killed for memory.

First checks

- Memory limits
- Usage
- Application behavior

```
> kubectl describe pod <pod>
> kubectl top pod <pod>
```

7 Node NotReady

Symptoms

- Node unhealthy, pods evicted.

First checks

- Node conditions
- Kubelet
- Disk/memory pressure
- Network

```
> kubectl describe node <node>
```

8 PVC Pending

Symptoms

- Volume claim stays pending.

First checks

- StorageClass
- PV availability
- Access mode
- Capacity

```
> kubectl describe pvc <pvc>
```

9 DNS Resolution Failure

Symptoms

- Service names do not resolve.

First checks

- CoreDNS pods/logs
- DNS config
- NetworkPolicy

```
> kubectl logs -n kube-system <coredns-pod>
```

10 High CPU/Memory

Symptoms

- Throttling, slow app, resource saturation.

First checks

- Top nodes/pods
- Requests/limits
- Replicas
- HPA

```
> kubectl top pods
```

QUICK COMMANDS

```
kubectl get pods -A | kubectl get events -A --sort-by=.lastTimestamp |
kubectl describe pod <pod> | kubectl logs <pod> --previous |
kubectl get nodes | kubectl top nodes | kubectl top pods
```

Production habit

1 → 2 → 3 → 4 → 5 → 6

Start with symptoms → check events →
inspect pod/node → check logs →
verify service/network → validate resources.



Golden rule: Do not jump straight to restarting. First capture state, events, logs, exit reasons, recent changes, and blast radius. Restarting can restore service, but it can also destroy the evidence needed to find the cause.

1. CrashLoopBackOff

Symptoms: A container starts, crashes, and is restarted repeatedly. Kubernetes increases the delay between restart attempts.

How to detect it

- `kubectl get pods -n <ns>`
- `kubectl describe pod <pod> -n <ns>` and inspect Last State / Exit Code / Events
- `kubectl logs <pod> -n <ns> --previous`
- Check readiness/liveness/startup probes, env vars, ConfigMaps, Secrets, command/args

Likely causes

- Application exception or bad startup command
- Missing/invalid config or secret
- Dependency unavailable during startup
- Probe kills the container before it becomes ready
- Permission/filesystem problem
- OOM or process exits intentionally

Fix / mitigation

9. Capture previous logs and exit code before changing anything.
10. Compare the failing pod template with the last known-good revision.
11. Fix the immediate cause: config, secret, image, probe timing, dependency, permission, or resource limit.
12. Roll out the corrected version or roll back to the last good revision.
13. Verify restart count stabilizes and readiness becomes true.

Useful commands

```
kubectl get pod <pod> -n <ns> -o wide
kubectl describe pod <pod> -n <ns>
kubectl logs <pod> -n <ns> --previous
kubectl rollout history deployment/<deploy> -n <ns>
kubectl rollout undo deployment/<deploy> -n <ns>
```

STAR incident example

S - Scenario	A newly deployed API begins restarting continuously and traffic drops.
T - Task	Restore a healthy replica set while preserving enough evidence to identify the crash.
A - Action	Checked previous logs and exit code, compared the new revision, found a missing required environment variable from a Secret, then rolled back and corrected the manifest.

R - Resolve

Service availability recovered after rollback; the corrected Secret reference was deployed with validation and a pre-deployment check was added.

2. ImagePullBackOff / ErrImagePull

Symptoms: The kubelet cannot download the container image. The pod remains unavailable and retries with backoff.

How to detect it

- `kubectl describe pod <pod> -n <ns>` and read Events
- Verify image repository and tag/digest
- Verify imagePullSecrets/service account
- Test registry reachability and authentication from the node/network path

Likely causes

- Wrong image name/tag
- Private registry credentials expired or missing
- Registry/network outage
- Architecture mismatch may surface later at runtime
- Rate limiting or policy restriction

Fix / mitigation

14. Correct repository/tag or pin a known-good digest.
15. Repair registry credentials or imagePullSecret attachment.
16. Restore network/registry access.
17. Recreate/rollout the workload and confirm the image pulls successfully.

Useful commands

```
kubectl describe pod <pod> -n <ns>
kubectl get pod <pod> -n <ns> -o jsonpath='{.spec.containers[*].image}'
kubectl get serviceaccount <sa> -n <ns> -o yaml
kubectl get secret -n <ns>
```

STAR incident example

S - Scenario	A production deployment scales up, but all new replicas remain ImagePullBackOff.
T - Task	Determine whether the release artifact or registry access is responsible and restore capacity.
A - Action	Read pod events, confirmed the image tag did not exist in the registry, reverted the deployment to the previous digest, and corrected the CI promotion step.
R - Resolve	Capacity returned immediately; CI was updated to validate image existence before deployment.

3. Pod Pending

Symptoms: A pod exists but has not been scheduled or cannot complete prerequisites required to start.

How to detect it

- `kubectl describe pod <pod> -n <ns>` and inspect scheduler Events
- Check node CPU/memory capacity and requests
- Check taints/tolerations, affinity/anti-affinity, topology spread
- Check PVC status and quotas
- Check node selectors and required labels

Likely causes

- Insufficient CPU/memory
- Taint not tolerated
- Affinity/zone constraints impossible to satisfy
- PVC not bound
- ResourceQuota/LimitRange issue
- No matching node selector

Fix / mitigation

18. Use scheduler event message to identify the binding constraint.
19. Adjust requests/constraints only if safe, or add capacity.
20. Fix storage binding or topology mismatch if PVC-related.
21. Verify pod schedules and becomes Ready.

Useful commands

```
kubectl describe pod <pod> -n <ns>
kubectl get nodes
kubectl top nodes
kubectl get pvc -n <ns>
kubectl get resourcequota -n <ns>
kubectl get events -n <ns> --sort-by=.lastTimestamp
```

STAR incident example

S - Scenario	A new worker deployment remains Pending after a scale-up.
T - Task	Identify the scheduling blocker without weakening production safety controls unnecessarily.
A - Action	Scheduler events showed insufficient memory on all eligible nodes. Confirmed requests, compared real usage, added node capacity, and later tuned requests from measured data.

R - Resolve

Pods scheduled successfully; capacity alerting and request-rightsizing were added to prevent repeat saturation.

4. Service Unreachable

Symptoms: Clients cannot connect to a Kubernetes Service, or requests time out/refuse.

How to detect it

- `kubectl get svc` and verify ports
- `kubectl get endpoints/endpointslices` and ensure ready backends exist
- Compare Service selector with pod labels
- Test pod IP directly, then Service DNS/IP
- Inspect NetworkPolicy and application listen address/port

Likely causes

- Selector mismatch
- Pods not Ready so no endpoints
- Wrong port/targetPort
- Application listening only on localhost or wrong port
- NetworkPolicy/CNI issue
- DNS problem

Fix / mitigation

22. Prove whether failure is DNS, Service routing, endpoint health, or application listener.
23. Fix selector/port/readiness/config or network policy.
24. Retest from an affected client pod.
25. Monitor errors and endpoint health after the fix.

Useful commands

```
kubectl get svc <svc> -n <ns> -o yaml
kubectl get endpoints <svc> -n <ns>
kubectl get endpointslice -n <ns> -l kubernetes.io/service-name=<svc>
kubectl get pods -n <ns> --show-labels
kubectl exec -it <client> -n <ns> -- curl -sv http://<svc>:<port>/health
```

STAR incident example

S - Scenario	An internal API times out even though its pods are Running.
T - Task	Restore service-to-service connectivity and isolate the failing layer.
A - Action	Checked Service endpoints and found none. Compared selector to pod labels and found a label changed in the deployment but not the Service. Updated the selector and verified endpoints populated.

R - Resolve

Traffic recovered immediately; label/selector consistency checks were added to deployment validation.

5. Ingress 502 / 503

Symptoms: External or ingress-routed requests return gateway errors while the ingress controller itself may still be healthy.

How to detect it

- Inspect Ingress rules and backend service/port
- Check Service endpoints and pod readiness
- Inspect ingress-controller logs/events
- Test backend Service directly from inside cluster
- Check TLS/certificate and protocol expectations

Likely causes

- No healthy endpoints
- Wrong service port/name
- Readiness failure
- Backend speaks HTTPS while ingress expects HTTP, or vice versa
- Timeouts from slow/unreachable backend
- Ingress controller/config issue

Fix / mitigation

26. Verify the backend independently of Ingress.
27. Fix service/endpoints/readiness/protocol configuration.
28. Reload/reconcile ingress if required.
29. Validate external request path and error rate after recovery.

Useful commands

```
kubectl describe ingress <name> -n <ns>
kubectl get svc,endpoints -n <ns>
kubectl logs -n <ingress-ns> <controller-pod>
kubectl exec -it <debug-pod> -n <ns> -- curl -sv http://<svc>:<port>/health
```

STAR incident example

S - Scenario	Users receive 503s immediately after an application rollout.
T - Task	Determine whether the ingress layer or backend availability is responsible.
A - Action	Tested the backend Service and found zero ready endpoints. Readiness probes were failing due to a changed health path. Corrected the probe path and rolled out the fix.

R - Resolve

Ready endpoints returned and 503s stopped; rollout checks were updated to block promotion when readiness never converges.

6. OOMKilled

Symptoms: The kernel/container runtime terminates a container because it exceeds its memory limit.

How to detect it

- `kubectl describe pod <pod>` and look for Reason: OOMKilled / exit code 137
- `kubectl top pod <pod> --containers`
- Compare requests/limits with historical usage
- Inspect application memory profile and recent traffic/change

Likely causes

- Memory leak
- Limit too low for legitimate workload
- Traffic spike
- Unbounded cache/batch size
- Too much concurrency
- Sidecar memory contention

Fix / mitigation

30. Stabilize by scaling out or temporarily raising limits when justified.
31. Identify whether usage is expected or leaking.
32. Tune application memory behavior/concurrency and right-size requests/limits.
33. Add alerts before memory reaches the kill threshold.

Useful commands

```
kubectl describe pod <pod> -n <ns>
kubectl top pod <pod> -n <ns> --containers
kubectl get pod <pod> -n <ns> -o jsonpath='{.spec.containers[*].resources}'
kubectl rollout status deployment/<deploy> -n <ns>
```

STAR incident example

S - Scenario	A worker begins restarting under peak load with exit code 137.
T - Task	Stop repeated OOMs while preserving throughput.
A - Action	Confirmed OOMKilled, graphed memory growth, reduced per-pod concurrency, increased replicas, and adjusted memory limits from measured peak usage.
R - Resolve	Restarts stopped and throughput stabilized; memory alerts and load tests were added.

7. NodeNotReady

Symptoms: A node stops reporting healthy status; workloads may be evicted or become unreachable.

How to detect it

- `kubectl get nodes`
- `kubectl describe node <node>` and inspect Conditions
- Check kubelet/runtime/network/disk/memory pressure
- Check cloud instance/system health and node-level logs
- Identify workloads placed on the affected node

Likely causes

- Kubelet stopped/unresponsive
- Network partition
- DiskPressure/MemoryPressure
- Container runtime failure
- Underlying VM/host failure
- CNI issue

Fix / mitigation

34. Cordon the node to stop new scheduling if instability is confirmed.
35. Drain when safe and capacity exists.
36. Repair or replace the node rather than repeatedly forcing workloads onto it.
37. Verify replicas reschedule and cluster capacity remains healthy.

Useful commands

```
kubectl get nodes
kubectl describe node <node>
kubectl get pods -A -o wide --field-selector spec.nodeName=<node>
kubectl cordon <node>
kubectl drain <node> --ignore-daemonsets --delete-emptydir-data
```

STAR incident example

S - Scenario	One worker node becomes NotReady and several application replicas disappear.
T - Task	Protect availability and move workloads away from the unhealthy node.
A - Action	Confirmed kubelet heartbeat loss, cordoned the node, verified spare capacity, drained affected workloads, and replaced the underlying instance.
R - Resolve	Replicas recovered on healthy nodes; node-health

alerting and capacity headroom were reviewed.

8. PVC Pending

Symptoms: A PersistentVolumeClaim cannot bind to suitable storage, blocking stateful pods.

How to detect it

- `kubectl describe pvc <pvc> -n <ns>`
- Check StorageClass name/default class
- Check requested size/access mode
- Check topology/zone requirements and provisioner events
- Check cloud/storage quotas and CSI driver health

Likely causes

- Missing/wrong StorageClass
- Unsupported access mode
- Provisioner/CSI failure
- Zone/topology mismatch
- Quota/capacity exhausted
- WaitForFirstConsumer interaction misunderstood

Fix / mitigation

38. Use PVC events to identify provisioning error.
39. Fix StorageClass, size, access mode, topology, or storage quota.
40. Verify PVC becomes Bound before expecting the pod to start.
41. Confirm application can mount/read/write the volume.

Useful commands

```
kubectl get pvc -n <ns>
kubectl describe pvc <pvc> -n <ns>
kubectl get storageclass
kubectl get pv
kubectl get pods -n kube-system | grep -i csi
```

STAR incident example

S - Scenario	A stateful service cannot start because its PVC remains Pending.
T - Task	Restore storage provisioning without risking existing data.
A - Action	Read PVC events, found the requested StorageClass did not exist in the target environment, mapped the workload to the supported class, and redeployed.
R - Resolve	PVC bound successfully and the pod started;

environment validation was added to deployment checks.

9. DNS Resolution Failure

Symptoms: Pods cannot resolve a Service or external/internal hostname, producing name-resolution errors before a connection is attempted.

How to detect it

- Test nslookup/dig/getent from an affected pod
- Check /etc/resolv.conf in the pod
- Check CoreDNS pods/logs/config
- Compare cluster-local names vs external/private DNS
- Check NetworkPolicy/egress to DNS and upstream resolvers

Likely causes

- CoreDNS unhealthy or overloaded
- Bad CoreDNS ConfigMap/stub domain
- Private DNS zone or forwarding missing
- Search domain/ndots confusion
- NetworkPolicy blocks DNS
- Upstream corporate/private resolver unreachable

Fix / mitigation

42. Determine whether only one name, one namespace, or all DNS is affected.
43. Repair CoreDNS/upstream forwarding/private zone association as appropriate.
44. Verify A/AAAA/CNAME response from the affected pod.
45. Then test TCP/TLS separately; DNS success does not guarantee service reachability.

Useful commands

```
kubectl exec -it <pod> -n <ns> -- nslookup <hostname>
kubectl exec -it <pod> -n <ns> -- cat /etc/resolv.conf
kubectl get pods -n kube-system -l k8s-app=kube-dns
kubectl logs -n kube-system -l k8s-app=kube-dns
kubectl get configmap coredns -n kube-system -o yaml
```

STAR incident example

S - Scenario	A workload cannot reach an internal corporate hostname and logs show “name or service not known.”
T - Task	Restore name resolution and determine whether the issue is cluster DNS or upstream private DNS.
A - Action	Reproduced the lookup from the pod, verified CoreDNS itself was healthy, then identified the private domain was not forwarded/associated with

the cluster network. Added the required private DNS forwarding/zone path and retested.

R - Resolve

The hostname resolved from the pod and application calls succeeded; a DNS validation check was added to the migration runbook.

10. High CPU / Memory

Symptoms: Pods are slow, throttled, unstable, or saturating resources even if they remain Running.

How to detect it

- `kubectl top pods/nodes`
- Compare usage to requests/limits
- Check HPA status and replica count
- Inspect latency/error metrics alongside CPU/memory
- Look for CPU throttling, GC pressure, queue growth, or hot partitions

Likely causes

- Traffic growth
- CPU limits too restrictive
- Memory leak/cache growth
- Insufficient replicas
- Inefficient query/workload
- Noisy neighbor or node saturation

Fix / mitigation

46. Protect service by scaling out or adding capacity.
47. Remove bottleneck based on evidence, not only by raising limits.
48. Right-size requests/limits from observed percentiles.
49. Add saturation and latency alerts tied to SLOs.

Useful commands

```
kubectl top pods -n <ns>
kubectl top nodes
kubectl get hpa -n <ns>
kubectl describe hpa <hpa> -n <ns>
kubectl get pod <pod> -n <ns> -o jsonpath='{.spec.containers[*].resources}'
```

STAR incident example

S - Scenario	API latency rises and CPU stays near the configured limit on every replica.
T - Task	Reduce customer impact while determining whether demand or a regression caused saturation.
A - Action	Correlated latency with CPU throttling, increased replicas, reviewed the recent release, and tuned a high-cost code path plus CPU limits/requests.
R - Resolve	Latency returned to target and HPA thresholds

were tuned using production metrics.

First 15 Minutes of a Kubernetes Incident

0-5 minutes: establish impact and preserve evidence

- What user-visible symptom is occurring?
- Which environment, namespace, service, region, and version are affected?
- Did a deployment/config/secret/network change happen recently?
- Capture pod states, events, restart counts, current/previous logs before restarting.

```
kubectl get pods -n <ns> -o wide
kubectl get events -n <ns> --sort-by=.lastTimestamp
kubectl rollout status deployment/<deploy> -n <ns>
kubectl describe pod <pod> -n <ns>
kubectl logs <pod> -n <ns> --previous
```

5-10 minutes: isolate the failing layer

- Application process: crash, health check, dependency, credentials?
- Service discovery: DNS resolution and Service endpoints?
- Network path: policy, route, firewall, ingress/load balancer?
- Resources: CPU, memory, node health, storage/PVC?

10-15 minutes: mitigate and verify

- Rollback if a known change correlates strongly with impact and rollback is safe.
- Scale out if saturation is the immediate cause and capacity exists.
- Fail over or bypass an unhealthy dependency when designed/approved.
- After mitigation, verify customer path, readiness, error rate, latency, and restart stability.

STAR Template for Reliability / Incident Stories

Use this format with managers and interviewers: Keep the technical depth, but organize it around business impact, your responsibility, evidence-driven actions, and measurable recovery/prevention.

S - Scenario	State the production symptom, affected users/services, environment, and why it mattered. Include severity or business impact if known.
T - Task	Describe what you were responsible for: triage, mitigation, coordination, root cause, or prevention. Avoid vague “we” when describing your own contribution.
A - Action	Walk through the evidence you gathered, hypotheses you ruled out, commands/metrics/logs you used, the mitigation you chose, and how you coordinated with other teams.
R - Resolve	Close with recovery and prevention: service restored, error/latency normalized, root cause identified, runbook/alert/test/policy added, and what reliability improved.

Example: DNS / private endpoint incident

S - Scenario	Pods in an EKS namespace could not resolve a required Cisco-internal hostname, blocking the workload from registering a downstream agent.
T - Task	Prove whether the fault was application, CoreDNS, VPC/private DNS, or network egress and restore the dependency path.
A - Action	Reproduced the failure from the pod, checked DNS resolver configuration and CoreDNS health, compared other resolvable names, validated the private-zone/forwarding path, then coordinated the DNS/network change and retested from the workload.
R - Resolve	Name resolution and application connectivity

recovered. The final runbook separated DNS validation from TCP/TLS validation and added a pre-cutover connectivity check.

Command Appendix

Cluster and workload overview

```
kubectl cluster-info
kubectl get nodes -o wide
kubectl get pods -A -o wide
kubectl get deploy,rs,svc,ingress -n <ns>
```

Events and object detail

```
kubectl describe pod <pod> -n <ns>
kubectl get events -n <ns> --sort-by=.lastTimestamp
kubectl get pod <pod> -n <ns> -o yaml
```

Logs

```
kubectl logs <pod> -n <ns>
kubectl logs <pod> -n <ns> --previous
kubectl logs <pod> -n <ns> -c <container> --since=30m
kubectl logs -f deployment/<deploy> -n <ns>
```

Rollouts

```
kubectl rollout status deployment/<deploy> -n <ns>
kubectl rollout history deployment/<deploy> -n <ns>
kubectl rollout undo deployment/<deploy> -n <ns>
kubectl rollout restart deployment/<deploy> -n <ns>
```

Networking

```
kubectl get svc,endpoints,endpointslice -n <ns>
kubectl exec -it <pod> -n <ns> -- nslookup <name>
kubectl exec -it <pod> -n <ns> -- curl -sv http://<service>:<port>/health
kubectl get networkpolicy -n <ns>
```

Resources

```
kubectl top pods -n <ns>
kubectl top nodes
kubectl get hpa -n <ns>
kubectl describe node <node>
```

Storage

```
kubectl get pvc,pv -n <ns>
kubectl describe pvc <pvc> -n <ns>
kubectl get storageclass
```

```
kubectl get volumeattachment
```

Config and secrets (metadata only)

```
kubectl get configmap -n <ns>  
kubectl describe configmap <name> -n <ns>  
kubectl get secret -n <ns>  
kubectl describe secret <name> -n <ns>
```

Security note: Do not print secret values into terminals, chat, tickets, or incident notes. Verify secret presence, key names, access path, and rotation status without exposing credentials.

Incident / RCA Template

Field	What to capture
Incident summary	One sentence describing the failure and impact.
Start / detection / recovery time	Include timezone and how the incident was detected.
Affected services/users	Blast radius and customer/business impact.
Symptoms	Errors, latency, restarts, failed dependencies, unavailable endpoints.
Recent changes	Deployments, config, secret, policy, DNS, infrastructure, scaling.
Timeline	Key observations and actions in chronological order.
Root cause	The technical condition that produced the incident.
Contributing factors	Gaps that increased likelihood or impact.
Mitigation	What restored service during the incident.
Permanent fix	What removes the root cause.
Prevention actions	Alerts, tests, SLOs, capacity, runbooks, automation, design changes.
Owner / due date	Accountability for each corrective action.

From Kubernetes Operator to Reliability Engineer

- **Reactive operations -> measurable reliability.** Track availability, latency, error rate, saturation, incident frequency, MTTR, and change-related failures.
- **Restarting -> diagnosis.** A restart can be a mitigation; reliability engineering asks why it was required and how recurrence will be prevented.
- **Infrastructure change -> service outcome.** Explain DNS, network, storage, secret, and deployment work in terms of customer/service reliability.
- **Individual fix -> system improvement.** Convert one-off fixes into alerts, safe defaults, tests, runbooks, policy, or automation.
- **Technical activity -> STAR impact story.** For every major incident, be able to explain the scenario, your task, your evidence-driven action, and the resolution/prevention.

Final takeaway: Kubernetes expertise becomes SRE value when it is used to detect risk early, reduce blast radius, restore service quickly, and prevent recurrence.